Rapport de Projet TP Compilation

Thème :

-Analyseur Lexicale de JavaScript .

-Analyseur Syntaxique de l’instruction Try-catch.

-Une interface pour tester les deux analyseurs.

Réaliser par : Mr.BENZAID Mohamed Amine

Section : A Groupe : 2

Année Universitaire : 2025-2026

جامعة بجاية
Tasdawit n Bgayet
Université de Béjaïa

RAPPORT DÉTAILLÉ SUR L'ANALYSEUR LEXICALE :

1. Introduction

L'objectif de ce programme est d'implémenter un **analyseur lexical (lexer)** capable de lire un programme JavaScript, de le découper en unités lexicales appelées **tokens**, et de classifier chacun de ces tokens selon sa nature : mots-clés, identifiants, opérateurs, chaînes, nombres, ponctuation, etc.

Cet analyseur est une étape essentielle dans la construction d'un **compilateur** ou d'un **interpréteur**, car il constitue la première phase d'analyse avant l'analyse syntaxique.

---

2. Structure générale du programme

Le programme est construit autour de trois éléments principaux :

1. **L'énumération TokenType**
   Elle définit les différents types de tokens possibles.
2. **La classe Token**
   Elle représente un token individuel avec son type, son lexème et sa position (ligne, colonne).
3. **La classe Lexer**
   Elle contient tout l'algorithme d'analyse lexicale.
4. **La méthode main**
   Elle teste le lexer avec un exemple complet de programme JavaScript.

---

3. La structure TokenType

```
enum TokenType {
  KEYWORD, IDENTIFIER, NUMBER, STRING,
  OPERATOR, PUNCTUATION, COMMENT,
  WHITESPACE, UNKNOWN, EOF
```

}

Cette énumération permet de classer chaque élément du code en un type bien défini.

Les types les plus importants sont :

- **KEYWORD** : mots-clés JavaScript (let, if, return, etc.)
- **IDENTIFIER** : noms de variables, fonctions, etc.
- **NUMBER** : nombres (entiers, flottants)
- **STRING** : chaînes de caractères (" ", ' ', ` `,)
- **OPERATOR** : opérateurs (+, ==, <=, &&, etc.)
- **PUNCTUATION** : ponctuation syntaxique ({, }, (,), ;, etc.)
- **COMMENT** : commentaires // ou /* */
- **WHITESPACE** : tous les espaces, tabulations, retours à la ligne
- **EOF** : fin de fichier '#'
- **UNKNOWN** : caractères non reconnus

---

4. La classe Token

Chaque token est représenté par un objet contenant :

- type : le type du token
- lexeme : la chaîne de caractères correspondant au token
- line et column : position dans le fichier

Cela permet d'avoir un excellent diagnostic d'erreurs.

Le toString() permet un affichage formaté pour le débogage.

---

5. La classe Lexer

C’est la partie la plus importante du programme.
Elle scanne le texte caractère par caractère et construit la liste des tokens détectés.

5.1 Variables internes

- input : tout le texte à analyser
- pos : index actuel dans le texte
- line, col : position dans le code source
- deux listes :
  - KEYWORDS
  - OPERATORS / PUNCTUATIONS

5.2 Méthodes utilitaires

- **estFin()** : vrai si on a atteint la fin du texte
- **regarder()** : renvoie le caractère courant sans l’avancer
- **regarderSuivant()** : renvoie le prochain caractère
- **avancer()** : avance d’un caractère et met à jour ligne/colonne

---

6. Détection des tokens

6.1 Les espaces

Les espaces, tabulations et retours à la ligne sont détectés comme :

TokenType.WHITESPACE

Ils sont conservés mais pourront être ignorés dans l’analyse syntaxique.

6.2 Les commentaires

Deux types :

- // commentaire
- /* commentaire multi-ligne */

Le code les détecte, les constitue en chaîne complète, puis les stocke en TokenType.COMMENT.

6.3 Les chaînes de caractères

Il reconnaît **trois types** :

- "..."
- '...'
- `...` (template strings JavaScript)

Il gère aussi les caractères échappés (\n, \", etc.).

6.4 Les nombres

Il détecte :

- les entiers : 123
- les flottants : 12.34
- les nombres scientifiques : 1.2e10

6.5 Les identifiants et mots-clés

Une séquence commençant par lettre, _ ou $.

Ensuite, s'il appartient à la liste KEYWORDS → TokenType.KEYWORD, sinon → TokenType.IDENTIFIER.

6.6 Les opérateurs

Le lexer reconnaît :

opérateurs à 3 caractères

===, !==

opérateurs à 2 caractères

<=, >=, ==, !=, =>, &&, ||, ++, --, etc.

opérateurs simples

+, -, *, /, <, >, !, =, etc.

6.7 Ponctuation

Caractères syntaxiques comme {, }, (,), ;, [,], ,, ..

6.8 UNKNOWN

Si un caractère ne correspond à rien, il est marqué comme inconnu → très utile pour repérer des erreurs.

---

7. La méthode analyserLexicale()

Cette méthode contient la boucle centrale qui :

1. lit le texte jusqu'à la fin
2. détecte le type du prochain token
3. ajoute ce token à la liste List<Token>
4. continue jusqu'à créer un dernier token EOF

C'est le cœur du programme.

---

8. Fonction main()

Elle :

1. crée un exemple de programme JavaScript
2. instancie le lexer
3. lance l'analyse
4. affiche les résultats

Elle montre tous les tokens détectés, et une seconde fois sans les espaces ni commentaires.

RAPPORT DÉTAILLÉ SUR L'ANALYSEUR LEXICALE :

1. Introduction

L'analyseur syntaxique (parser) constitue la deuxième étape principale d'un compilateur ou interpréteur, après l'analyse lexicale.
Son rôle est de vérifier que la séquence de tokens produite par l'analyseur lexical respecte les règles grammaticales du langage et de détecter toute erreur structurelle.

La classe AnalyseurSyntaxique2 implémente un analyseur récursif descendant pour un sous-ensemble inspiré de JavaScript, gérant :

- les déclarations de variables
- les affectations
- les classes et méthodes
- les expressions arithmétiques et logiques
- les blocs try/catch/finally
- certaines instructions ignorées (while, for, if)

2.La Grmmaire officielle Associée à l'anlayseur syntaxique :

1-Programme :

Le programme est une séquence d'instructions jusqu'au token EOF.

Programme → Instruction Programme | ε

2-Instructions :

Priorité :

1. var | let | const

2. ident = … (affectation)
3. class …
4. try … catch … (finally …)
5. while/for/if ignorés

Instruction → DeclarationVar | Affectation | DeclarationClass | TryCatch | InstructionIgnoree

3-Variables :

DeclarationVar → (var | let | const) IDENTIFIER DeclarationVarSuite ;

DeclarationVarSuite → = Expression | ε

4-Affectation :

Affectation → IDENTIFIER = Expression ;

5-Expressions :

SuiteComparaison permet :

- opérations arithmétiques : + - * / %
- comparateurs : == != === !== < > <= >=
- opérateurs logiques : && ||

Expression → Primaire SuiteComparaison

SuiteComparaison → OPERATEUR Primaire SuiteComparaison | ε

Primaire → IDENTIFIER | NUMBER | STRING

6-Déclaration de classe :

DeclarationClasse → class IDENTIFIER { ListeMethodes }

ListeMethodes → Methode ListeMethodes | ε

Methode → IDENTIFIER ( ) { }

Le parser accepte seulement :

- méthodes sans paramètres
- corps vide

7-Structure try / catch / finally :

TryCatch → try Bloc catch (IDENTIFIER) Bloc TryFinally

TryFinally → finally Bloc | ε

8-Bloc :

Utilisé spécifiquement pour :

- méthodes
- try
- catch
- finally
- autres blocs

Bloc → { ListeInstructions }

ListeInstructions → Instruction ListeInstructions |

9- Instructions ignorées :

InstructionIgnoree → while | for | if

3. Architecture Générale

L'analyseur fonctionne selon le principe du *recursive descent parsing* :

- Chaque règle de la grammaire est implémentée par une méthode Java.
- Le token courant est stocké dans courant.
- La méthode avancer() passe au token suivant.
- Les méthodes match() et type() vérifient la présence de tokens spécifiques.
- skipInsignificant() élimine les espaces et commentaires.
- En cas d'erreur, la méthode erreur() produit un message précis avec ligne et colonne.

---

4. Gestion des erreurs syntaxiques

L'analyseur signale avec précision :

- la ligne
- la colonne
- le token trouvé
- ce qui était attendu

Petite description de l'interface :

L'interface de l'analyseur est conçue pour être simple et intuitive. Elle permet à l'utilisateur de saisir un programme JavaScript dans une zone de texte dédiée, del MonProgramme.jarpuis de lancer l'analyse syntaxique à l'aide d'un bouton. Les résultats de l'analyse, qu'il s'agisse d'un message de réussite ou d'erreurs détectées, s'affichent clairement dans une zone prévue à cet effet. L'interface facilite ainsi l'utilisation de l'analyseur en offrant un environnement convivial et facile à comprendre, même pour les utilisateurs débutants.